



ASHOK IT

Learn Here.. Lead Anywhere...!!

GENERATIVE AI & AGENTIC AI

Student Study Notes & Application Architecture

Prepared for Ashok IT LMS

A practical and interview-oriented guide to LLMs, prompting, RAG, vector databases, LangChain, LangGraph, CrewAI, MCP and cloud deployment.

Learn Here.. Lead Anywhere...!!

Course Roadmap

A structured learning path from Gen AI fundamentals to production-oriented Agentic AI.

#	Module	Focus
1	Gen AI & Agentic AI	Generation, agents, goals, tools and state.
2	Gen AI Application Architecture	UI, backend, orchestration, models and tools.
3	Large Language Models	Tokens, prompts, inference and model providers.
4	Hosted vs Local LLMs	Paid APIs versus local/self-hosted inference.
5	OpenAI GPT Project	Build a hosted-LLM application.
6	Ollama Project	Run and integrate a local LLM.
7	Prompt Engineering	Templates, zero-shot, few-shot and structured reasoning.
8	RAG Architecture	Ground LLM responses with external knowledge.
9	Vector Databases	Embeddings and semantic similarity search.
10	RAG Application	Build document question answering.
11	LangChain	Compose prompts, models, retrievers and tools.
12	LangGraph	Build stateful graph-based workflows.
13	CrewAI	Build role-based multi-agent systems.
14	MCP	Connect AI applications with tools/resources.
15	Cloud Deployment	Secure, deploy and monitor Gen AI applications.
16	Interview Guidance	Explain projects, architecture and trade-offs.

1. What is Generative AI?

Generative AI creates new content from instructions, learned patterns and supplied context.

Definition: Generative AI is a class of AI systems that can generate new content such as text, code, images, audio, video and structured outputs.

Common outputs:

- Text — explanations, emails, documentation and summaries.
- Code — functions, SQL, tests and application scaffolding.
- Images — illustrations, concepts and marketing assets.
- Audio and video — speech, music and synthetic media.
- Documents — extraction, classification, summarization and question answering.

Example: User Prompt → Gen AI Model → Generated Response

Traditional AI: Student Data → ML Model → Pass / Fail

Gen AI: User Prompt → Gen AI Model → Explanation / Code / Summary

2. Traditional AI vs Generative AI

Traditional / Predictive AI	Generative AI
Usually predicts a label, score or value from input data.	Generates new content from instructions and context.
Example: Student data → ML model → Pass / Fail.	Example: User prompt → LLM → Explanation / Code / Summary.
Often optimized for prediction accuracy.	Often optimized for useful, coherent and context-aware generation.

Example prompt: Explain Spring Boot in simple terms.

Possible answer: Spring Boot is a Java framework that helps developers build applications faster with sensible defaults, auto-configuration and less manual setup.

3. What is Agentic AI?

Agentic AI extends Gen AI with goals, decisions, tools, state and multi-step execution.

- **Goal:** define the desired outcome.
- **Decision-making:** determine what action should happen next.
- **Tools:** APIs, databases, search, code execution or enterprise services.
- **State:** retain relevant information across workflow steps.
- **Execution:** act → observe → decide → continue or stop.

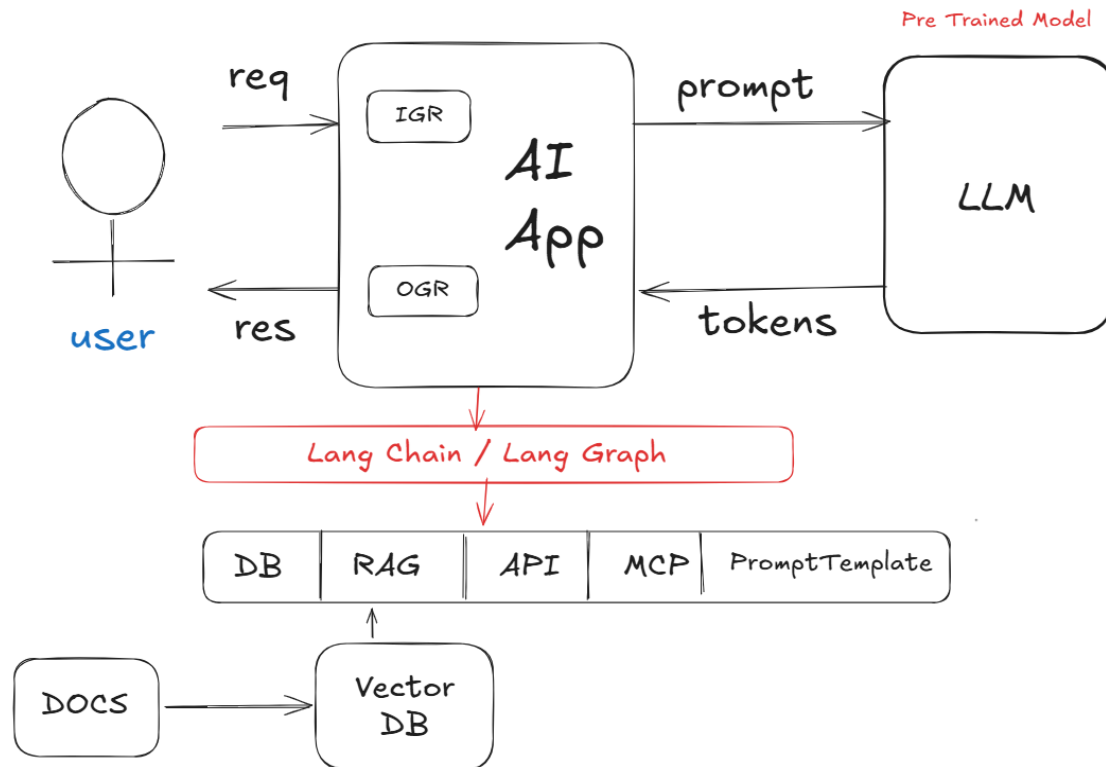
Simple distinction: Gen AI mainly generates content. Agentic AI uses a model inside a controlled workflow that decides which actions to perform to reach a goal.

Gen AI	Agentic AI
Prompt → Model → Response	Goal → Decide → Tool/Action → Observe → Next Action → Result
Primarily generation	Goal-oriented task execution
Often one main generation step	Often multi-step and iterative

4. Gen AI Application Architecture

An enterprise Gen AI application is more than an LLM.

The application receives a user request, prepares the prompt, invokes a pre-trained LLM and returns the generated result. The orchestration layer can also connect the model to retrieval, databases, APIs, MCP tools, prompt templates and application state.



Architecture layers: User Interface → AI Application / Orchestrator → LLM, RAG, Databases, APIs and Tools.

5. What is an LLM?

LLM = Large Language Model

An LLM is a machine-learning model trained on very large collections of text and code so that it can model language patterns and generate sequences of tokens.

Examples: OpenAI GPT, Google Gemini, Meta Llama and Anthropic Claude.

- **Token:** a unit of text processed by the model.
- **Prompt:** instructions and context supplied to the model.
- **Inference:** generating output from a trained model.
- **Context window:** the amount of context the model can process for a request.
- **Temperature:** controls generation variability; lower values generally produce more deterministic output.

An LLM is not simply a fixed-answer database. It generates output from learned patterns and the context supplied at inference time. This is why grounding, validation and guardrails are important.

6. Hosted / Paid LLM vs Local LLM

Choose the deployment model based on quality, cost, privacy, latency and operational requirements.

For teaching, distinguish commercial hosted APIs from locally running open-weight models. A local model may have no per-request provider fee, but hardware, electricity, GPU memory and maintenance still have costs.

Aspect	Hosted / Paid API	Local / Self-hosted
Execution	Provider infrastructure	Developer or organization infrastructure
Access	API key + SDK / HTTP	Local runtime / API endpoint
Cost	Usually usage-based	No provider API fee; infrastructure costs remain
Setup	Fast to start	Requires runtime and model setup
Privacy	Depends on provider and service configuration	Can keep inference inside controlled infrastructure
Examples	OpenAI GPT, Gemini, Claude APIs	Llama-family and other supported local models

Hosted GPT flow: User → UI → Backend → Prompt Template → LLM API → Response

Ollama flow: User → UI → Backend → Prompt → Ollama Runtime → Local LLM → Response

7. Prompt Engineering

Design prompts to make model output more useful, consistent and controllable.

A strong prompt usually specifies the **role**, **task**, **context**, **constraints** and **expected output format**.

Prompt Template

A reusable prompt contains placeholders for dynamic values.

```
You are a Java trainer.  
Explain {topic} for a beginner.  
Use simple language, one real-world example and five interview questions.  
Return the answer using headings and bullet points.
```

Zero-Shot Prompting

Ask the model to perform a task without providing examples.

```
Classify this review as Positive, Negative or Neutral: "The course content is useful."
```

Few-Shot Prompting

Provide a small number of examples so the model can infer the expected pattern.

```
Input: Great product → Sentiment: Positive  
Input: Very poor support → Sentiment: Negative  
Input: The product is okay → Sentiment: ?
```

Chain-of-Thought / Structured Reasoning

Complex tasks may benefit from structured intermediate work. In production, prefer concise plans, verifiable intermediate outputs and explicit checks rather than exposing private internal reasoning.

8. RAG Architecture

RAG = Retrieval-Augmented Generation

RAG combines retrieval from an external knowledge source with LLM generation. It is useful when answers need private, domain-specific or frequently changing information.

Core RAG flow: Documents → Chunking → Embeddings → Vector DB → Query → Similarity Search → Context → Prompt → LLM → Answer

- **Documents:** PDFs, Word files, web pages, manuals, policies and knowledge bases.
- **Chunking:** split documents into smaller, meaningful pieces.
- **Embedding:** convert each chunk into a numerical vector.
- **Vector database:** store vectors together with metadata.
- **Retriever:** find relevant chunks for the query.
- **Prompt assembly:** combine user question and retrieved context.
- **LLM:** generate a grounded answer.

9. Vector Databases

Term	Meaning
Embedding	Numerical representation of content for semantic comparison.
Vector Database	System optimized for storing and searching vectors, often with metadata filters.
Similarity Search	Find vectors mathematically close to the query vector.
Metadata	Source, page, category, timestamp and other attributes.

Semantic retrieval can find relevant content even when the wording of the query differs from the wording in the source document.

10. Gen AI Application with RAG

Think of a RAG application as two related pipelines.

Ingestion Pipeline	Query Pipeline
Load documents	Receive user question
Parse / clean content	Create query embedding
Split into chunks	Search vector database
Create embeddings	Retrieve top relevant chunks
Store vectors + metadata	Build grounded prompt
Repeat when knowledge changes	Call LLM and return answer

Quality controls: choose appropriate chunk sizes, preserve metadata, tune top-k retrieval, filter irrelevant content, ground answers in retrieved evidence and evaluate retrieval separately from generation.

11. LangChain Introduction

LangChain provides an ecosystem for composing LLM application components.

- Prompt templates for reusable instructions.
- Chat model integrations for different providers and runtimes.
- Document loaders and text splitters for RAG pipelines.
- Retrievers for semantic knowledge access.
- Tools for APIs, databases and external actions.
- Runnable pipelines / chains for composing multiple steps.
- Agent abstractions for tool selection and iterative execution.

Mental model: LangChain acts as an orchestration layer connecting application logic with models, prompts, retrieval and tools.

12. Agentic AI with LangChain

An agent can select an appropriate tool based on the user's goal. Example: "Check my order and tell me the delivery date." → agent selects order-status API → receives result → produces final response.

- Define the model and system instructions.
- Register tools with clear names, descriptions and input schemas.
- Allow tool selection when necessary.
- Execute the selected tool and return its observation.
- Continue until the task is complete or a safe stopping condition is reached.

Production caution: Tool permissions, input validation, timeouts, retries, logging and human approval may be required for high-impact actions.

13. LangGraph Introduction

Graph-based orchestration is useful for stateful, branching and iterative agent workflows.

LangGraph is designed for building stateful agent workflows as graphs. Developers can model nodes, edges, conditional transitions and shared state explicitly.

- **State:** information carried across workflow steps.
- **Node:** a unit of work such as an LLM, retriever or tool call.
- **Edge:** determines the next step.
- **Conditional edge:** routes execution based on state or a result.
- **Checkpointing:** can support resumable workflows when configured and designed for that purpose.

Example: START → Understand → Retrieve → Decide → Tool → Validate → Retry / Retrieve More → Final Response → END

14. Multi-Agent Systems with CrewAI

Divide a larger task among specialized agents when specialization adds value.

A multi-agent system gives different agents separate roles, goals, tools and expected outputs.

Agent	Responsibility
Researcher	Find important topics, skills and project ideas.
Writer	Convert research into structured, student-friendly content.
Reviewer (optional)	Check completeness, correctness, tone and formatting.

Example: Course Content Generator → Researcher → Writer → Reviewer → Final Content

Use multiple agents when specialization provides a real benefit. Do not add agents merely to make an architecture more complex.

15. MCP — Model Context Protocol

MCP provides a standardized way for AI applications to connect with external tools and resources through a defined protocol.

- **MCP Server:** exposes tools and resources.
- **MCP Client:** the AI application or agent connecting to the server.
- **Tool:** an action the client can invoke.
- **Resource:** data or context exposed to the client.

Flow: User → AI App / Agent → MCP Client → MCP Server → Tool / Resource → Result → Agent → User

Key benefit: standardized connectivity can reduce one-off integrations and make agent capabilities easier to reuse.

16. Cloud Deployment

Apply standard software-engineering practices plus Gen AI-specific cost, security and observability controls.

- **UI:** Streamlit, React, Angular or another frontend.
- **Backend:** FastAPI, Flask, Spring Boot or another API framework.
- **Model:** hosted LLM API or self-hosted inference.
- **Knowledge:** object storage + document processing + vector database.
- **Data:** SQL / NoSQL database as required.
- **Security:** secret manager, authentication, authorization and validation.
- **Observability:** logs, traces, latency, token usage, retrieval quality and error metrics.

Typical deployment: User → Web UI → Backend API → Orchestration → LLM / RAG / Tools → Databases & External Services

17. Interview Guidance

Focus on architecture, trade-offs and your actual implementation decisions.

- What is Generative AI? How is it different from traditional ML?
- What is an LLM and what are tokens?
- Explain zero-shot and few-shot prompting.
- What is RAG and why is it required?
- Explain chunking, embeddings, vector databases and similarity search.
- How would you reduce hallucinations in a RAG application?
- LangChain vs LangGraph — when would you use each?
- What is an AI agent and what makes an application agentic?
- How do tools and APIs fit into an agent workflow?
- What is a multi-agent system and when is it useful?
- What is MCP? Explain MCP client and MCP server.
- How would you secure and monitor a production Gen AI application?

18. Project Explanation Template

Use this structure when explaining a Gen AI project in an interview.

Area	What to explain
Problem	What user or business problem did the project solve?
Architecture	UI → Backend → Orchestrator → LLM / RAG / Tools.
Data	What documents, databases or APIs were used?
Model	Why was the selected model/runtime appropriate?
Prompt	How were instructions and output format designed?
Retrieval	How were documents chunked, embedded and retrieved?
Evaluation	How was answer quality measured?
Security	How were keys, permissions and sensitive data protected?
Deployment	Where and how was the application deployed?

Strong interview answer pattern: Problem → Architecture → Technology Choice → Implementation → Challenges → Trade-offs → Result.

19. Quick Revision

A compact recap for last-minute revision.

Concept	Remember
Gen AI	Generates new content from prompts and context.
Agentic AI	Uses goals, decisions, tools and state for multi-step tasks.
LLM	Large model trained to process and generate language/token sequences.
Prompt	Instruction + context + constraints + expected output.
RAG	Retrieve external knowledge and provide it to the LLM as context.
Embedding	Numerical representation used for semantic comparison/search.
Vector DB	Stores and searches embeddings, often with metadata filters.
LangChain	Orchestration for models, prompts, retrieval and tools.
LangGraph	Graph-based, stateful orchestration for complex workflows.
CrewAI	Role/task-oriented multi-agent orchestration.
MCP	Protocol for connecting AI clients with tools/resources.
Ollama	Local runtime for running supported LLMs on your infrastructure.

Final architecture mindset: Start with the simplest solution that works. Add **RAG** when external knowledge is needed, **tools** when actions are needed, **state** when workflow context must persist, **graphs** when orchestration becomes complex, and **multiple agents** only when specialization provides measurable value.

Ashok IT — Generative AI & Agentic AI Student Notes

www.ashokit.in | +91-9985396677